



**Universidad
Nacional
de San Martín**

UNIVERSIDAD NACIONAL DE SAN MARTÍN

EXAMEN PARCIAL DE PYTHON

PROGRAMACIÓN ORIENTADA A OBJETOS, REQUESTS Y ARCHIVOS

Instrucciones

- Lee cuidadosamente cada consigna antes de programar.
- El examen tiene una duración de **2 horas**.
- Se evaluarán: funciones, comprensiones, uso de decoradores, requests, programación orientada a objetos y manejo de archivos y excepciones.
- Se descontarán puntos por errores de sintaxis, falta de claridad o no cumplir con el formato pedido de input y output.

1. Ejercicio 1 – Sistema de Usuarios y Decoradores (50 pts)

Se desea implementar un pequeño sistema para gestionar usuarios y sus permisos en una aplicación web.

1. (Lambda y comprensiones – 8 pts) A partir de una lista de diccionarios con la siguiente estructura:

```
1 usuarios = [  
2     {"nombre": "Ana", "rol": "admin", "edad": 34},  
3     {"nombre": "Luis", "rol": "editor", "edad": 28},  
4     {"nombre": "Marta", "rol": "lector", "edad": 41},  
5     {"nombre": "Juan", "rol": "admin", "edad": 22},  
6 ]
```

Usa:

- Una expresión **lambda** para filtrar solo los usuarios administradores.
- Una **comprensión de listas** para obtener la lista de nombres de estos usuarios.

—

2. (Clases y POO – 12 pts) Implementa una clase **Usuario** con:

- Atributos: **nombre**, **rol**, **edad**.
- Un método **presentarse()** que devuelva un string con el formato: "Soy Ana y mi rol es admin".

Luego, genera una lista de instancias de **Usuario** a partir de la lista original.

—

3. (Pilares de la POO – 10 pts) Extiende el sistema con las siguientes clases:

- **UsuarioBasico**: solo puede leer información.
- **UsuarioEditor**: puede leer y editar.
- **UsuarioAdmin**: puede leer, editar y eliminar.

Requisitos:

- Usa **herencia** para no repetir código.
- Sobrescribe un método **acciones_permitidas()** para demostrar **polimorfismo**.

—

4. (Decoradores – 10 pts) Implementa un **decorador** llamado **@restringir_admin** que solo permita ejecutar una función si el **rol** del usuario es **admin**. En caso contrario, debe lanzar una excepción personalizada **PermisoDenegadoError**.

—

5. (Archivos y Excepciones – 10 pts) Implementa una función **guardar_usuarios(usuarios)** que:

- Guarde en un archivo **usuarios.txt** cada usuario en una línea con el formato **nombre,rol,edad**.
- Use manejo de excepciones para capturar errores de escritura y mostrar un mensaje claro en caso de fallo.

2. Ejercicio 2 – Generadores y API de Clima (50 pts)

Se desea construir un pequeño módulo para consultar y procesar datos del clima utilizando la API pública de Open-Meteo (<https://api.open-meteo.com>).

1. **(Requests – 10 pts)** Realiza una solicitud HTTP GET a la API de Open-Meteo para obtener el pronóstico de temperatura horaria en Buenos Aires (lat: -34.61, lon: -58.38) para el día actual. URL base:

`https://api.open-meteo.com/v1/forecast?latitude=-34.61&longitude=-58.38&hourly=temperature\_2m`

Guarda la respuesta en un archivo `clima.json`. Implementa manejo de excepciones para errores de conexión.

2. **(Archivos y Excepciones – 10 pts)** Carga el archivo `clima.json` y procesa la lista de temperaturas horarias. Implementa control de excepciones para:

- Archivo no encontrado
 - Errores de decodificación JSON
-

3. **(Generadores – 10 pts)** Implementa un **generador** llamado `filtrar_calor(temperaturas, umbral)` que reciba la lista de temperaturas y un umbral (por ejemplo, 25°C), y **vaya devolviendo solo las temperaturas mayores al umbral**.
-

4. **(Lambda y comprensiones – 10 pts)** Usando:

- Una expresión `lambda` para redondear las temperaturas a enteros.
 - Una **comprensión de diccionarios** para construir un diccionario `hora: temperatura` con las primeras 10 horas.
-

5. **(Clases + Decoradores – 10 pts)** Crea una clase `ReporteClimatico` con:

- Atributos: `fecha`, `temperaturas`.
- Un método `resumen()` que imprima la temperatura máxima, mínima y promedio.

Implementa un decorador `@formatear_salida` que envuelva el método `resumen()` para mostrar un encabezado bonito antes de imprimir el resultado.